

Guide Complet : Les Génériques en Java

Paramétrage des types, Sécurité de type, Wildcards et Exercices Pratiques

Introduction aux Génériques

Introduits en Java 5, les **génériques** permettent de paramétrer les classes, interfaces et méthodes avec des types (classes ou interfaces).

Pourquoi les utiliser ?

- **Sécurité de type (Type Safety)** : Détection des erreurs de type à la *compilation* plutôt qu'à l'exécution (`ClassCastException`).
- **Suppression du casting** : Plus besoin de convertir explicitement les objets récupérés d'une collection.
- **Réutilisabilité** : Écrire du code agnostique vis-à-vis du type de données manipulé.

Syntaxe et Conventions

Par convention, les paramètres de type utilisent une seule lettre majuscule :

- `<T>` : Type général (Type)
- `<E>` : Élément (souvent utilisé dans les Collections)
- `<K, V>` : Clé et Valeur (pour les Maps)
- `<N>` : Nombre

Classes et Méthodes Génériques

1. Classe Générique

On définit un paramètre de type après le nom de la classe :

```
public class Boite<T> {
    private T contenu;

    public void setContenu(T contenu) {
        this.contenu = contenu;
    }

    public T getContenu() {
        return contenu;
    }
}
```

2. Méthode Générique

Le paramètre de type se place avant le type de retour de la méthode :

```
public class Utilitaire {
    public static <T> void echanger(T[] tab, int i, int j) {
        T temp = tab[i];
        tab[i] = tab[j];
        tab[j] = temp;
    }
}
```

Bornes de Types (Bounded Type Parameters)

Il est possible de restreindre les types acceptés à l'aide du mot-clé **extends**.

- **Borne supérieure** : `<T extends Number>` n'autorise que les classes filles de `Number` (ex : `Integer`, `Double`).

```
public class Stats<T extends Number> {
    private T[] valeurs;
    // Permet d'appeler des methodes de Number (ex: doubleValue())
}
```

Les Jokers (Wildcards : ?)

Le point d'interrogation `?` représente un type inconnu.

- **Wildcard non borné** : `List<?>` (accepte n'importe quelle liste).
- **Wildcard covariant (extends)** : `List<? extends Number>` (lecture seule, PECS : Producer Extends).
- **Wildcard contravariant (super)** : `List<? super Integer>` (écriture favorisée, PECS : Consumer Super).

Exemple Global

```
import java.util.List;

public class TestGeneriques {
    // Methode utilisant un wildcard covariant
    public static double somme(List<? extends Number> liste) {
        double total = 0.0;
        for (Number n : liste) {
            total += n.doubleValue();
        }
        return total;
    }

    public static void main(String[] args) {
        Boite<String> boiteChaine = new Boite<>();
        boiteChaine.setContenu("Bonjour Java");
        System.out.println(boiteChaine.getContenu());

        List<Integer> entiers = List.of(1, 2, 3, 4);
        System.out.println("Somme : " + somme(entiers));
    }
}
```

Exercices Pratiques

Exercice 1 : Paire générique (Niveau Facile)

1. Créez une classe générique `Paire<K, V>` possédant deux attributs : `cle` (de type `K`) et `valeur` (de type `V`).
2. Ajoutez un constructeur, les getters, les setters et une méthode `toString()`.
3. Dans un programme principal, instanciez une `Paire<String, Integer>` représentant par exemple un nom d'article et son prix.

Exercice 2 : Méthode de recherche (Niveau Moyen)

1. Écrivez une méthode générique statique nommée `compterOccurrences` prenant un tableau d'éléments de type `T` et un élément cible de type `T`.
2. La méthode doit retourner le nombre de fois que l'élément apparaît dans le tableau.
3. Testez cette méthode avec un tableau de chaînes de caractères et un tableau d'entiers.

Exercice 3 : Boîte sécurisée et bornes (Niveau Avancé)

1. Créez une classe `CalculatriceTab<T extends Number>` qui prend un tableau de `T` en attribut.
2. Écrivez une méthode `double obtenirMoyenne()` qui retourne la moyenne des éléments du tableau sous forme de `double`.
3. Testez avec un tableau de `Integer` et un tableau de `Double`.